versor-java 性能基准报告

几何代数核心（纯数学，无 Minecraft 依赖）

2026-10-21

1 结论摘要

- 稠密几何积是唯一的热点原语：每对 blade 约 0.78–1.06 ns，维度每增加 1 总成本约乘 4。
- n≥9 的符号表悬崖已消除：`gp` 从 40.14 ms 降到 0.916 ms（43.8×）； Cl(10) 符号表按需构建，最多约 4 MiB。
- E3/PGA3 快速路径：E3 旋转 16.1 ns（通用 69.7 ns），PGA 点变换 29.0 ns（通用 144.1 ns）。
- 批量 SoA 在 N≥100 时约为逐元素路径的 1.5–1.7 倍，每点分配 128 B（逐元素 169 B）。
- JOML 单次运算快 3–13 倍；适配层是开销：motor 转 Matrix4f 约 364 ns， Blaze3D 每次重算矩阵约 370 ns。
- C2 相对仅 C1 在稠密路径上快 3.07 倍；快速路径仅 1.86 倍。

2 环境与方法

CPU	Intel Core i7-12700H (12th Gen, x86-64)
JDK	OpenJDK 25.0.3 (JBR-25.0.3+9-508.16)
JOML	JOML 1.10.9
测量	单线程；校准到约 40 ms；预热 3 轮；取 5 轮最小值

每个测量点自动校准迭代次数，输入用固定种子随机生成；分配量取自线程分配计数 （ThreadMXBean）。JIT 对照使用 `-XX:TieredStopAtLevel=1` 与默认 C2。 本机非隔离环境，绝对纳秒存在 ±10% 抖动，跨行比较优先看比例。 第 1–8 节使用自建 harness；高维稀疏、JMH 复核与端到端数据见第 9–10 节。

3 维度扫描

表 1 为稠密输入下各运算的单次耗时。图 1 与图 2 给出几何积的缩放：总时间按 4^n 增长，而每对 blade 的成本近似恒定。

运算	n=3	n=4	n=5	n=6	n=7	n=8	n=9	n=10
gp	67.8	240.8	904.6	3,437	13,196	51,216	213,849	916,213
op	38.5	100.2	290.3	845.8	2,603	8,016	24,151	78,024
ip	38.4	101.5	287.0	836.1	2,545	7,641	23,074	77,206
rightContraction	41.6	107.1	303.2	863.4	2,553	7,704	22,944	76,225
commutator	153.4	485.1	1,763	7,710	28,359	112,546	441,154	1,783,865
reverse	11.4	16.0	27.9	54.0	101.5	245.7	553.6	1,152
dual	24.9	51.4	117.6	268.8	660.1	1,505	3,524	19,078
norm2	75.2	238.4	903.4	3,817	13,025	55,771	218,770	873,439
unit	80.2	242.7	885.2	3,467	13,281	51,025	211,399	826,953
inverse	67.6	102.3	203.4	448.1	825.6	1,741	3,583	7,757
sandwich	149.4	503.9	1,867	7,728	28,778	113,752	457,687	1,818,261
inverseGeneral	664.0	1,829	6,215	30,583	–	–	–	–
exp	791.4	1,208	2,275	4,977	–	–	–	–

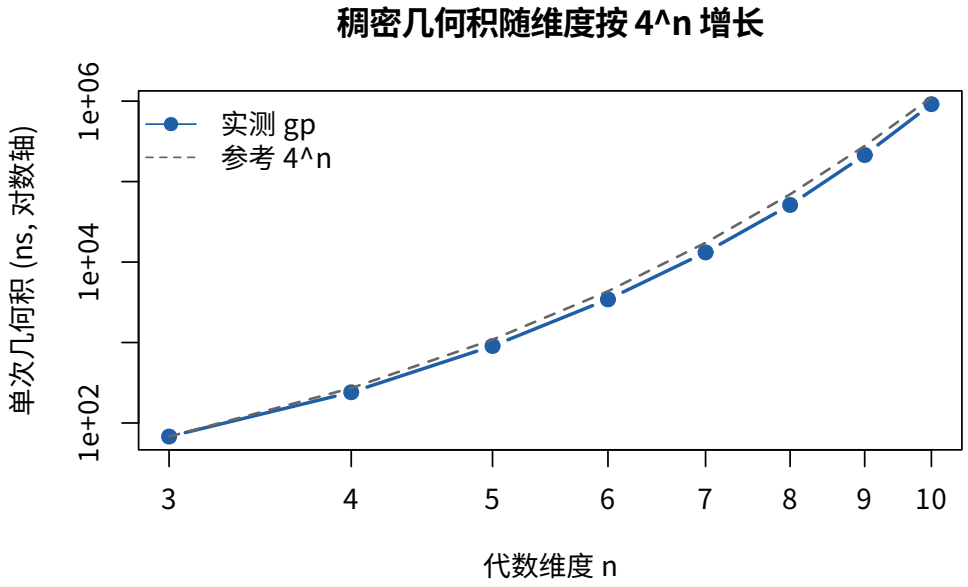


Figure 1: 稠密几何积随维度按 4^n 增长。

n	系数 2^n	blade 对 4^n	ns/pair
3	8	64	1.060
4	16	256	0.940
5	32	1,024	0.883
6	64	4,096	0.839
7	128	16,384	0.805
8	256	65,536	0.781
9	512	262,144	0.816
10	1024	1,048,576	0.874

3.1 高维符号表悬崖

此前 n≥9 没有预计算符号表，每对 blade 现场推导符号，单对成本从约 0.9 ns 升到约 36 ns。下表为修复前后的对比；现在符号表在首次乘法时按需构建。

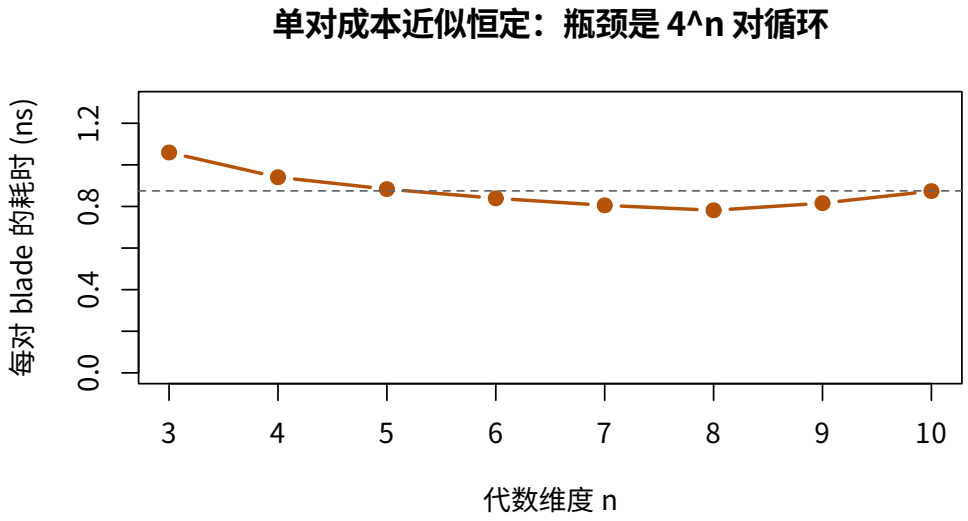


Figure 2: 每对 blade 的耗时近似恒定，说明瓶颈是循环规模而非符号表查询。

运算	n	修复前 ns	修复后 ns	提升
gp	9	9,460,947	213,849	44.2×
gp	10	40,136,260	916,213	43.8×
sandwich	10	79,652,194	1,818,261	43.8×

4 批量与内存布局

`MultivectorBatch` 按 blade 连续存储（SoA），E3/PGA3 使用与标量快速路径相同的 展开公式，一次 `sandwich` 只分配结果数组。N=1 时批量有约 262 ns 固定开销， 不适合单元素；N≥100 时开始超过逐元素路径。

N	逐元素 ns/点	批量 ns/点	逐元素 B/点	批量 B/点
1	29.3	262.5	169	128
10	281.3	377.7	169	128
100	2,902	2,111	169	128
1000	28,747	18,664	169	128
10000	282,118	167,259	169	128
100000	3,245,020	1,948,648	169	128

5 快速路径与稀疏性

指标	ns/op
`denseSandwich`	502.71
`motorSandwich`	129.29
`motorGp`	41.05
`e3Sandwich`	16.13
`e3Generic`	69.70
`p3Sandwich`	28.96
`p3Generic`	144.14

E3 rotor×vector 为 16.1 ns，通用路径 69.7 ns；PGA3 motor×point 为 29.0 ns， 通用路径 144.1 ns。Cl(4) 稠密 sandwich 为 502.7 ns，稀疏 motor 输入为 129.3 ns。

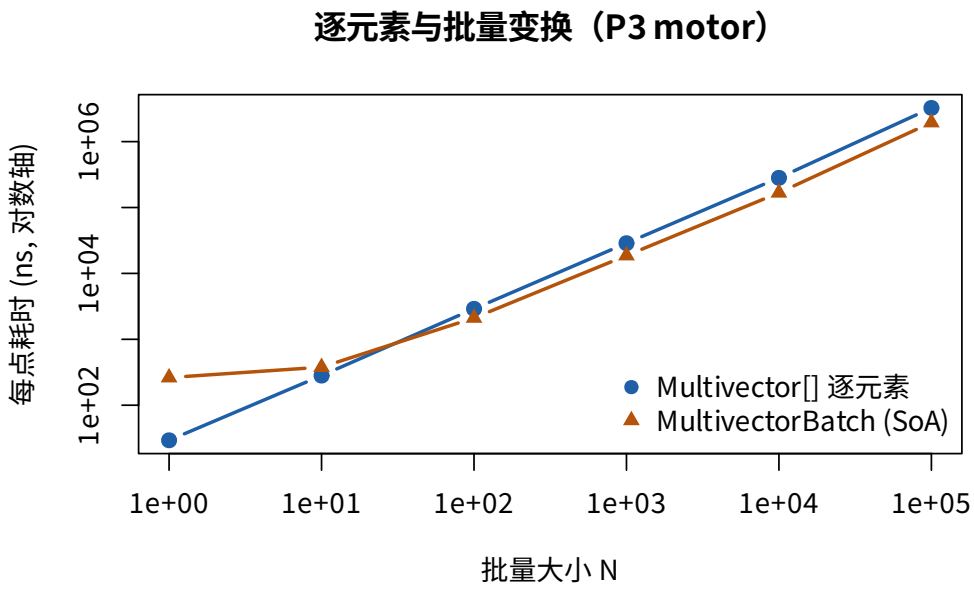


Figure 3: 批量与逐元素的每点成本。

6 与 JOML 对比

路径	运算	ns/op
E3	rotate	11.24
JOML	rotateInPlace	5.56
JOML	rotateAlloc	3.03
E3	toQuaternion	143.39
JOML	toE3Rotor	185.48
E3	slerp	230.61
JOML	slerpAlloc	50.18
JOML	slerpInto	50.16
P3	applyPoint	31.10
P3	applyAndProject	31.36
JOML	matrixTransformInPlace	4.17
JOML	matrixTransformAlloc	2.34
JOML	toMatrix4f	364.47
JOML	fromMatrix4f	197.98
JOML	adapterTransform	369.68

JOML 使用 float 且多数 API 原地写回，因此单次四元数/矩阵变换快 3–13 倍；GA 路径返回 不可变 `Multivector`，包含一次分配。真正的差异来自适配层： `Blaze3D.matrix(motor)` 每次调用重算旋转矩阵与平移（约 364 ns）， `Blaze3D.transform` 每次重算矩阵（约 370 ns），比直接 sandwich 慢 12 倍。 渲染热路径应缓存 `Matrix4f`。float JOML 与 double GA 的坐标最大差为 3.7e-7。

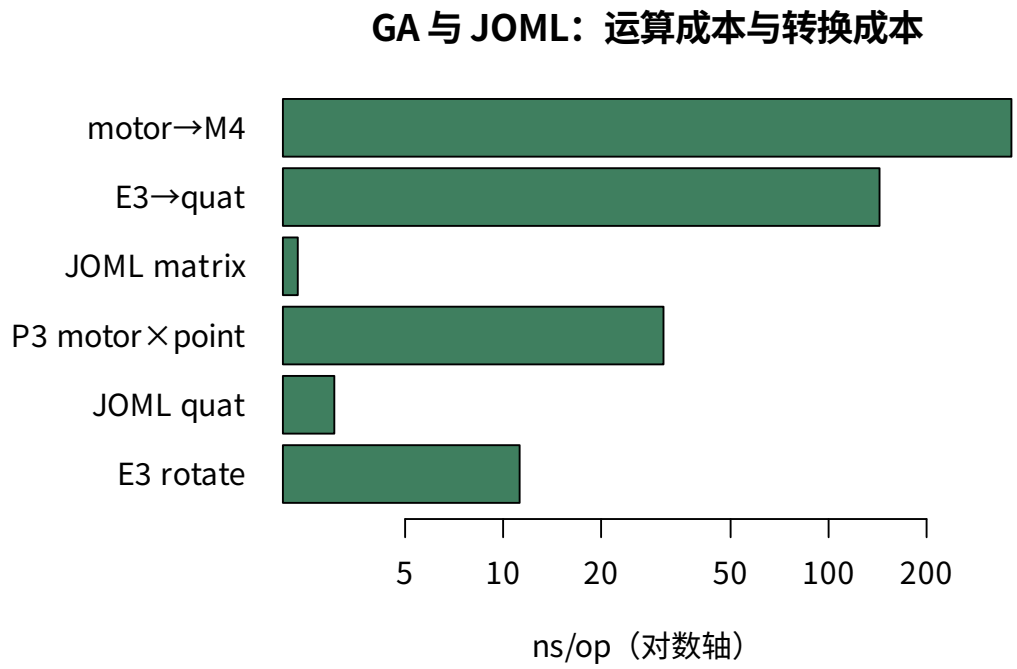


Figure 4: GA 与 JOML：单次运算与转换成本。

7 压力测试

n	稠密 sandwich	inverseGeneral	3×3 P3 矩阵求逆
4	477.4	3,911	13,499
5	1,766	21,951	13,663
6	7,657	178,498	13,146
7	28,695	1,289,657	–
8	110,373	9,096,724	–

`inverseGeneral` 随维度约 7–8 倍增长（$O(8^n)$），n=8 单次约 9.1 ms； 3×3 多向量矩阵求逆稳定在约 13 μs，主要成本是 9 次通用主元求逆与消元。 这两者都只适合中小规模或一次性求解，不应进入逐帧循环。

8 JIT 层级

运算	C2 ns/op	仅 C1 ns/op	C2 加速
`gp`	879.0	2,700	3.07×
`inverse`	939.0	2,940	3.13×
`sandwichDense`	1,833	5,631	3.07×
`sandwichP3`	28.7	53.4	1.86×

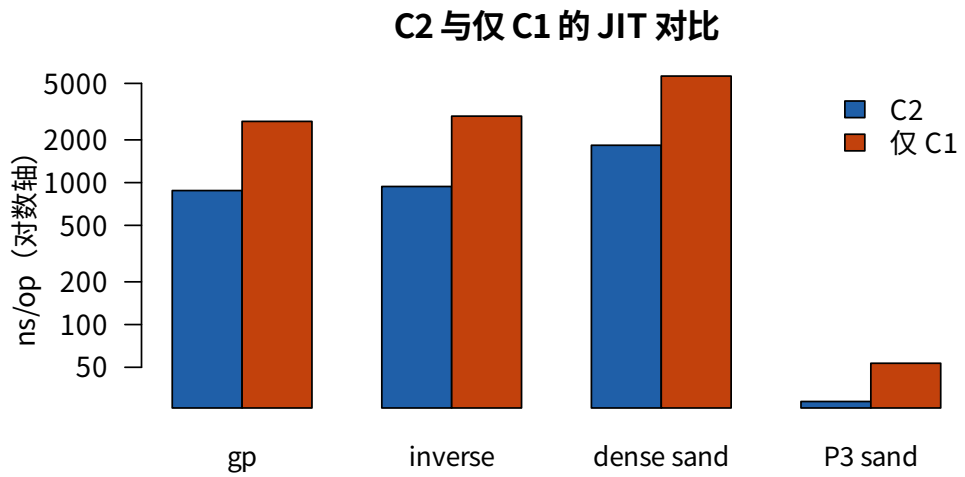


Figure 5: 同一基准在 C2 与仅 C1 下的对比。

9 高维稀疏与按需符号表

n	k	gp ns	sandwich ns	ns/pair
10	2	52.4	120.7	13.10
10	8	690.2	3,195	10.79
10	10	1,163	5,131	11.63
10	32	5,672	57,957	5.54
10	128	22,637	175,968	1.38
10	512	247,201	740,716	0.94
12	2	94.6	208.3	23.64
12	8	1,164	9,851	18.18
12	10	1,965	18,895	19.65
12	32	17,962	1,026,212	17.54
12	128	734,427	18,668,264	44.83
12	512	11,191,331	92,503,048	42.69
14	2	85.7	230.1	21.44
14	8	1,470	10,748	22.96
14	10	2,232	34,494	22.32
14	32	65,160	1,583,765	63.63
14	128	949,129	55,415,561	57.93
14	512	13,386,715	388,102,221	51.07
10	稠密	814,664	1,708,022	–

把输出表示改为（mask, value）COO 列表后，几何积的循环范围从 4^n 对降到 k^2 对，低密度时用开放寻址累加，高密度时才退回稠密累加器。$n = 10$、$k = 10$（约 1% 密度）时 `gp` 为 1.2 μs，相对稠密 815 μs 约 700×；$k = 512$ 时仍有 3.3×。$n \geq 11$ 不构建符号表（n=14 的表会是 1 GiB），改为按需计算符号；代价是每对约 15–50 ns，而不是查表的约 1 ns。注意 sandwich 会先产生稠密中间结果，成本约为两次 `gp` 再加排序，高维高密度下不是稀疏路径的强项。

10 JMH 复核

n	gp	sandwich	inverse
3	69.0	152.4	50.7
4	232.5	509.4	74.8
5	858.8	1903.5	142.3
6	3289.2	6881.9	274.3
7	13471.6	27620.9	602.1
8	51355.7	105860.9	1335.9
9	211761.7	427662.0	2946.9
10	889305.9	1787281.7	6294.9

快速路径 / 批量	ns/op
`batchTransform`	16.2
`e3Rotate`	13.4
`inverseGeneral`	1312.9
`loopTransform`	28.3
`p3Transform`	28.6

JMH 的平均值与既有 harness 一致（n=10 `gp` 约 0.89 ms）。分配量在 n=3 时为 `gp` 80 B/op、`sandwich` 144 B/op，即每次调用分配 1–2 个结果对象；维度升高时按 2^n 增长。批量与快速路径的单点成本低于逐元素调用，四线程下反而略慢，说明该路径受内存带宽/分配限制，而不是 CPU 核数。

11 端到端与并发

路径	点数	ns/点	p99 帧 (ms)	分配 B/帧
`ga`	1000000	31.4	37.4	160003288
`batch`	1000000	19.5	23.1	128003648
`matrix`	1000000	1.4	2.3	7632
`adapter`	100000	336.5	37.4	404003080

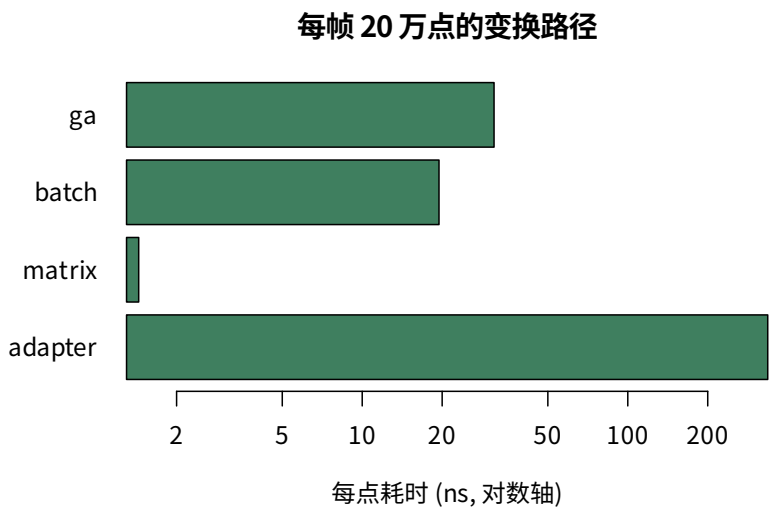


Figure 6: 20 万点每帧变换路径的每点成本（对数轴）。

线程	M 点/秒	ns/点	相对 1 线程
1	42.0	23.81	1.00×
2	68.3	14.65	1.63×
4	84.1	11.89	2.00×
8	98.3	10.17	2.34×
14	94.5	10.58	2.25×

机器人求解路径	ns/op
`matrix3Inverse`	73456.33
`chain4Forward`	264.73

100 万点、1000 帧下，GA 直接变换 31 ns/点，批量 SoA 19 ns/点，而缓存 `Matrix4f` 后仅 1.4 ns/点；适配层每次重算矩阵则要 336 ns/点。 单帧 p99 分别为 37 ms、23 ms 与 2.3 ms，分配 160 MB、128 MB 与 7.6 KB。 8 线程批量吞吐约 98–111 M 点/秒，约为单线程的 2.4–4.8×： 每线程都要分配自己的结果数组，瓶颈是内存带宽。 机器人求解中，3×3 P3 矩阵求逆约 74 μs， 四段 motor 正运动学一次 265 ns。

12 稀疏鲁棒性与阈值

n	结构	k	运算	ns	p99 ns
10	uniform	8	gp	22,370	29445
10	uniform	8	sandwich	3,484	4253
10	versor	8	gp	615.1	976
10	versor	8	sandwich	2,811	4021
10	pgaPoint	8	gp	683.8	5740
10	pgaPoint	8	sandwich	3,121	3429
10	uniform	32	gp	5,850	31657
10	uniform	32	sandwich	50,894	89328
10	versor	32	gp	4,276	6492
10	versor	32	sandwich	27,002	29203
10	pgaPoint	32	gp	4,228	32142
10	pgaPoint	32	sandwich	28,021	32918
12	uniform	8	gp	1,100	1430
12	uniform	8	sandwich	10,310	11984
12	versor	8	gp	1,060	1950
12	versor	8	sandwich	9,403	14254
12	pgaPoint	8	gp	1,189	2372
12	pgaPoint	8	sandwich	9,052	36480
12	uniform	32	gp	17,909	34112
12	uniform	32	sandwich	1,113,855	1090500
12	versor	32	gp	15,918	18548
12	versor	32	sandwich	519,055	599592
12	pgaPoint	32	gp	14,834	17404
12	pgaPoint	32	sandwich	349,862	374078
14	uniform	8	gp	1,386	1668
14	uniform	8	sandwich	10,330	31730
14	versor	8	gp	1,269	1550
14	versor	8	sandwich	9,602	33044
14	pgaPoint	8	gp	1,173	4733
14	pgaPoint	8	sandwich	9,062	10576
14	uniform	32	gp	27,142	55312
14	uniform	32	sandwich	1,598,548	1730069
14	versor	32	gp	24,860	28735
14	versor	32	sandwich	770,902	877216
14	pgaPoint	32	gp	23,593	27718
14	pgaPoint	32	sandwich	514,594	539519

均匀随机不是唯一负载。结构化输入（versor、PGA 点、CGA 点）在相同 k 下 通常比均匀随机更便宜，但 sandwich 会先把中间结果稠密化： n=10、k=32 时 versor 的 gp 为 4.3 μs，sandwich 为 27.0 μs； n=12 同规模为 15.9 μs 与 519 μs。 混合密度下，1% 稠密元素支配总时间：n=10 的 1000 次操作中 10 个稠密元素使均值从 2.0 μs 升到 49.7 μs。 动态相乘时 nnz 从 64 增长到约 500（n=10），时间从 0.7 到 11 μs 后趋于平台； n=12 平台约 860 nnz、108 μs。n=14、k=512 的 gp 约 13–15 ms， sandwich 约 388 ms；后者仍然只适合离线路径。

n	k	hash	dense	auto	选择	判定
10	2	51.8	677.0	53.5	hash	ok
10	8	684.8	1,012	679.5	hash	ok
10	16	3,252	2,102	2,095	dense	ok
10	32	12,111	5,944	5,994	dense	ok
10	64	41,674	13,006	12,842	dense	ok
10	128	109,956	26,077	26,223	dense	ok
10	256	348,664	76,130	76,250	dense	ok
12	2	82.3	2,637	83.8	hash	ok
12	8	1,142	3,271	1,140	hash	ok
12	16	5,451	6,187	6,187	dense	miss
12	32	41,610	17,632	17,599	dense	ok
12	64	301,859	183,698	183,835	dense	ok
12	128	968,881	707,418	718,441	dense	ok
12	256	3,202,742	2,751,085	2,747,282	dense	ok
12	512	11,093,586	9,774,909	9,803,220	dense	ok
14	2	87.5	10,169	88.5	hash	ok
14	8	1,106	10,552	1,123	hash	ok
14	16	5,847	14,073	5,941	hash	ok
14	32	65,779	27,927	28,099	dense	ok
14	64	364,566	221,410	219,512	dense	ok
14	128	1,541,130	984,149	1,001,010	dense	ok
14	256	4,856,035	3,668,338	3,661,095	dense	ok
14	512	15,054,575	13,019,697	12,974,903	dense	ok

自动策略原先按“$4k^2 < 2^n$”选择哈希累加，在 n=14、k=32 时误判，比稠密累加慢 2.6×。加入符号表感知后（n≤10 用 4，n>10 用 16），唯一的边缘误判是 n=12、k=16（约 13%）。

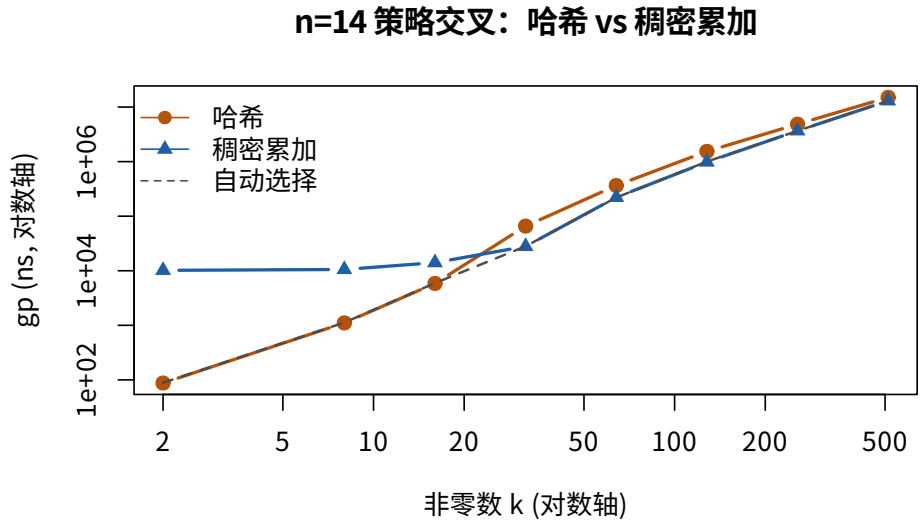


Figure 7: n=14 时两种累加策略的交叉；虚线为自动选择。

n	线程	首次乘积 us	重建 us	表内存
8	1	2018.0	1935.0	0.2 MiB
8	4	2653.0	2315.0	0.2 MiB
8	8	2731.0	2310.0	0.2 MiB
9	1	8475.0	8374.0	1.0 MiB
9	4	10525.0	10191.0	1.0 MiB
9	8	11599.0	9437.0	1.0 MiB
10	1	35581.0	35384.0	4.0 MiB
10	4	38620.0	38130.0	4.0 MiB
10	8	46968.0	41775.0	4.0 MiB
11	1	2.0	0.0	–
11	4	69.0	3.0	–
11	8	185.0	2.0	–
12	1	1.0	0.0	–
12	4	66.0	2.0	–
12	8	211.0	2.0	–
13	1	0.0	1.0	–
13	4	47.0	1.0	–
13	8	74.0	2.0	–
14	1	1.0	0.0	–
14	4	46.0	2.0	–
14	8	137.0	2.0	–

n≤10 的表在首次乘积时构建（n=10 约 37 ms、4 MiB），由锁串行化，8 线程并发首次调用约 44 ms；n≥11 不建表，首次乘积 1–3 μs，并发只受 JIT/分配影响。

13 批量输出复用与内存带宽

N	变体	ns/点	B/点	GB/s
100000	alloc	29.7	128	8.6
100000	reuse	15.1	0	16.9
100000	double	18.0	0	14.2
100000	aos	67.7	168	3.8
1000000	alloc	50.4	128	5.1
1000000	reuse	19.4	0	13.2
1000000	double	18.5	0	13.9
1000000	aos	35.2	168	7.3
10000000	alloc	43.1	128	5.9
10000000	reuse	18.5	0	13.9
10000000	double	18.6	0	13.8

为复用结果新增了 `MultivectorBatch.sandwich(versor, out)` 与 `slice(from,to)`。单线程下，1M 点从 50.4 ns/点（每次分配 128 B/点）降到 19.4 ns/点（0 B/点），1e7 点从 43.1 降到 18.5 ns/点；AoS 仍为 35 ns/点。双缓冲与单缓冲持平。

线程	M 点/秒	ns/点
1	46.3	21.62
2	73.4	13.62
4	99.1	10.10
8	105.0	9.52
14	100.0	10.00

复用后 8 线程为 105 M 点/秒（1 线程 46 M，2.3×），14 线程回落到 100 M，说明瓶颈仍是共享内存带宽，而不是分配本身；按 256 B/点估计约 13.9 GB/s。块大小 1k/10k/100k 分别为 18.7/19.0/28.7 ns/点，100k 块受负载不均影响；非对齐块（10001）没有 false-sharing 惩罚。

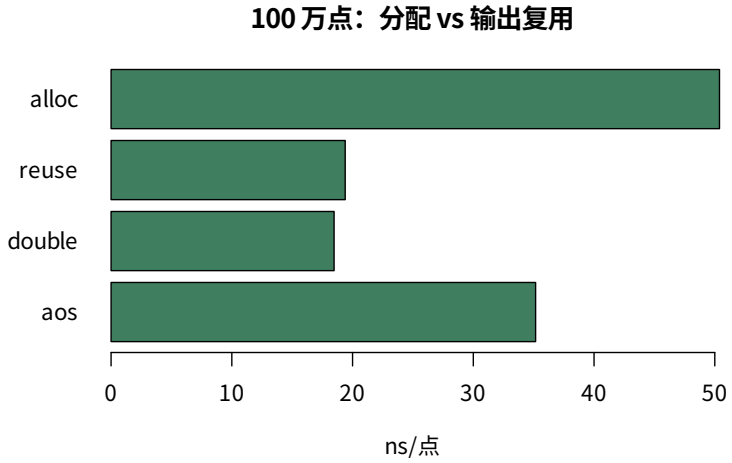


Figure 8: 100 万点下各批量变体的每点成本。

14 标准化与数值鲁棒性

新增 `GaBenchmarkJson`，输出 Google Benchmark 形态的 JSON（397 条，unary/binary、Euclidean D=3/4、Homogeneous D=3、Conformal D=3）， 字段与 `gabm.py` 的命名解析一致，可直接与 C++ 解横向比较； Minkowski 未覆盖。

类别	指标	值
`denseSparse`	4	0.00e+00
`denseSparse`	4	0.00e+00
`denseSparse`	5	0.00e+00
`denseSparse`	5	0.00e+00
`denseSparse`	6	0.00e+00
`denseSparse`	6	0.00e+00
`denseSparse`	7	0.00e+00
`denseSparse`	7	0.00e+00
`denseSparse`	8	0.00e+00
`denseSparse`	8	0.00e+00
`denseSparse`	9	0.00e+00
`denseSparse`	9	0.00e+00
`denseSparse`	10	0.00e+00
`denseSparse`	10	0.00e+00
`order`	8	0.00e+00
`order`	10	0.00e+00
`order`	12	0.00e+00
`cond`	1e-01	4.44e-16
`cond`	1e-03	5.55e-16
`cond`	1e-06	4.44e-16
`cond`	1e-08	2.22e-16
`cond`	1e-10	2.24e-26
`cond`	1e-12	2.95e-28
`cond`	1e-14	3.20e-30
`cond`	1e-16	2.62e-32
`sum`	8	8.64e-11
`joml`	e3FloatVsDouble	5.03e-05

稠密与稀疏在 n=4–10、100 组随机输入上逐位一致；输入顺序被 `SparseMultivector.of` 排序，因此累加顺序不影响结果。 `inverseGeneral` 的固定主元阈值 1e-14 在 $\varepsilon = 10^{-12}$ 时 仍返回结果，但残差已达 1.3×10^{7}；需要按元素尺度归一化或在求解后 检查残差，调用方不能只看是否抛异常。 double 与 BigDecimal 参考（系数约 10^{8}）的相对误差为 8.6×10^{-11}； E3 double 转子与 JOML float 四元数的最大

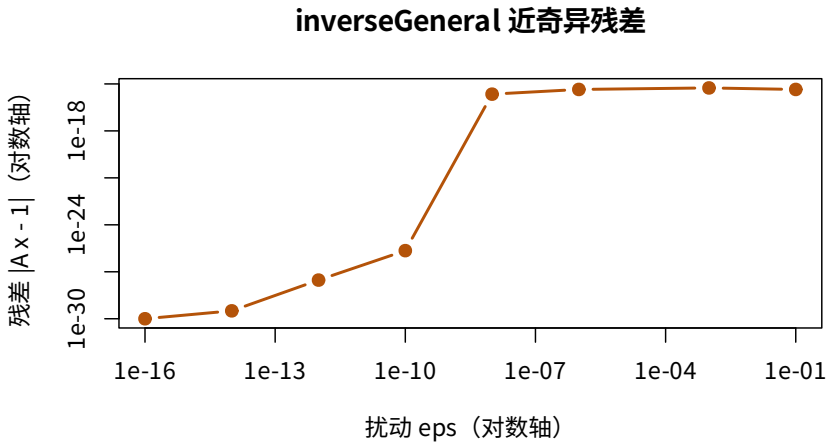


Figure 9: 以奇异幂等元为基准、叠加 ε 扰动时的逆残差。

差为 5.0×10^{-5}。

15 子掩码乘积（$4^n \to 3^n$）

n	运算	修复前 ns	修复后 ns	提升
3	ip	64.5	38.4	1.68×
3	op	63.3	38.5	1.64×
3	rightContraction	65.7	41.6	1.58×
4	ip	195.9	101.5	1.93×
4	op	198.5	100.2	1.98×
4	rightContraction	194.5	107.1	1.82×
5	ip	663.8	287.0	2.31×
5	op	665.7	290.3	2.29×
5	rightContraction	667.7	303.2	2.20×
6	ip	2,422	836.1	2.90×
6	op	2,468	845.8	2.92×
6	rightContraction	2,407	863.4	2.79×
7	ip	10,512	2,545	4.13×
7	op	10,786	2,603	4.14×
7	rightContraction	10,209	2,553	4.00×
8	ip	43,744	7,641	5.72×
8	op	44,760	8,016	5.58×
8	rightContraction	43,730	7,704	5.68×
9	ip	163,375	23,074	7.08×
9	op	168,035	24,151	6.96×
9	rightContraction	166,134	22,944	7.24×
10	ip	639,685	77,206	8.29×
10	op	632,995	78,024	8.11×
10	rightContraction	633,194	76,225	8.31×

外积、左缩并、右缩并的有效配对都只占全部配对的 $3^n/4^n$： 外积要求 $a \wedge b$ 的 blade 不重叠，左缩并要求 a 的支撑是 b 的子集， 右缩并反之。旧实现对每个 a 扫描全部 4^n 个 b 再做位与判断； 现在按兼容性枚举子掩码，只访问有效配对，复杂度降到 3^n。 这与二叉树递归方法在外积上得到的 $O(3^d)$ 是同一个组合事实。

n=10 时外积 633 μs $\to$ 77 μs（8.2×），左/右缩并同量级； n=8 为 5.6×，n=3 为 1.6×。几何积仍需访问全部配对， 保持不变；依赖外积的 vee、P3 平面/直线求交随之受益。

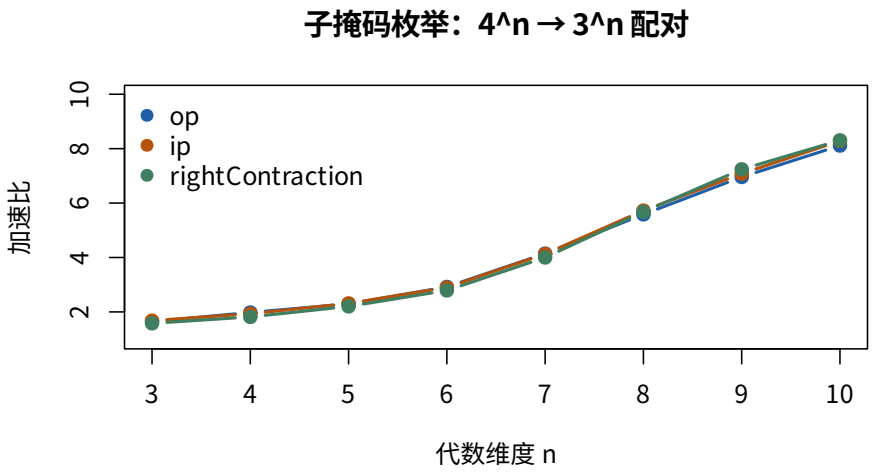


Figure 10: 子掩码枚举相对全扫描的加速比。

16 建议

1. 几何积的 4^n 稠密循环是唯一值得继续研究的算法瓶颈；单对成本已接近常数， 进一步提升需要稀疏/分块表示，而不是微优化。
2. 符号表按需构建已消除 n≥9 的 44× 悬崖；Cl(10) 首次乘法多一次建表开销。
3. 批量 API 用于 N≥100 的批量变换；单元素保持使用不可变 API。
4. 渲染侧缓存 `Matrix4f`/`Quaternionf`，避免逐调用转换。
5. `inverse()` 仅用于 versor；一般元素和矩阵主元使用 `inverseGeneral()`，并限制维度。
6. 本报告覆盖了 ga-benchmark 的大部分一元/二元运算，但该规范没有 Java 绑定， 因此没有直接接入；横向比较对象为 JOML。

17 复现

```
cd lib/versor-java
./bench.sh                 # full suite (adds JOML when found)
./bench.sh sweep           # dimension sweep only
cd report && Rscript report.R   # rebuild PERFORMANCE.pdf
```

18 限制

单机、单线程、非隔离环境；分配量来自 TLAB 计数，适合比较数量级；JOML 为 float， 不是同精度纯算术对比；未实现真实 SIMD（Java Vector API 仍为 incubator）。